

Homework 01

Template assignment

Student Name · Student ID

COURSE 101 · Course Name

Month DD, YYYY

Instructor Name · Collaborators: None

Problem 1: A recurrence and its closed form

10 points

Consider the sequence defined by

$$a_0 = 1, \quad a_n = 2a_{n-1} + 1 \quad \text{for } n \geq 1.$$

- (a) Compute the first four terms.
- (b) Prove that $a_n = 2^{n+1} - 1$ for every $n \geq 0$.

 **Solution** The first four terms are 1, 3, 7, and 15. We prove the closed form by induction. The base case is $a_0 = 1 = 2^1 - 1$. If $a_k = 2^{k+1} - 1$, then

$$a_{k+1} = 2a_k + 1 = 2(2^{k+1} - 1) + 1 = 2^{k+2} - 1.$$

Therefore, the formula holds for all $n \geq 0$.

Problem 2: Complexity analysis

5 points

The following algorithm performs a linear search. State its worst-case running time and justify your answer.

 linear_search.py

```
1 def locate(values, target):
2     for index, value in enumerate(values):
3         if value == target:
```

```
4         return index
5     return None
```

 **Solution** In the worst case, the algorithm inspects every element exactly once. Its running time is therefore $\Theta(n)$ and its auxiliary space usage is $\Theta(1)$. The source and explanation are kept together following the general *literate-programming* principle described by [Knuth \(1984\)](#).

References

Donald E. Knuth. *Literate programming*. *The Computer Journal*, 27(2):97–111, 1984. doi: 10.1093/comjnl/27.2.97.